IA em Finanças: Previsão, Fraudes, Drift e Agentes de IA

Resumo da conversa · 1º de novembro de 2026

Ideia central: a IA funciona muito bem em detecção de fraude (sinal forte, alvo bem definido) e muito pior em previsão de preços (sinal fraco, ruído alto, regimes que mudam). Em ambos, manter o modelo relevante ao longo do tempo (drift) é o problema que mais pesa na prática. Agentes de IA (LLMs com ferramentas) ajudam a organizar a análise e a justificar decisões, mas o controle de risco deve ficar em código, fora do LLM. Os capítulos 6 a 11 trazem diagramas e código (Python para os agentes, Go para o backend) e o glossário está no final.

1. Como a IA prevê preços e detecta fraudes

Previsão de preços de ações

Prever o preço exato de amanhã quase nunca é o objetivo. Mercados são quase eficientes, e um modelo que acerta a direção em 52–55% das vezes já pode ser lucrativo, mas esse edge some com custos, slippage e overfitting. Na prática prevê-se retorno relativo entre ativos, volatilidade, probabilidade de alta/baixa e regime de mercado.

- **Gradient boosting (XGBoost, LightGBM):** cavalo de batalha em dados tabulares (retornos passados, volume, indicadores, fundamentos).
- **Séries temporais:** ARIMA/GARCH (GARCH é padrão para volatilidade), LSTM, GRU, Transformers temporais (TFT, PatchTST).
- **NLP:** sentimento extraído de notícias, balanços e comunicados vira feature.
- **Aprendizado por reforço:** aprende política de compra/venda; promissor, mas instável.

Como aprende: minimiza uma função de perda mapeando features para o alvo. O ponto crítico é a validação: usar divisão temporal (`TimeSeriesSplit`), nunca aleatória, para evitar *look-ahead bias*. Em finanças sérias: walk-forward, purging/embargo e backtest com custos realistas.

Armadilhas: vazamento de dados, overfitting ao backtest, não-estacionariedade e *survivorship bias*.

Detecção de fraudes

- **Supervisionada:** classificador (GBDT domina) sobre transações rotuladas; features como valor, horário, localização, dispositivo, frequência e desvio do comportamento habitual.
- **Não supervisionada:** Isolation Forest, autoencoders, One-Class SVM, úteis com poucos rótulos ou fraudes novas.
- **Grafos (GNNs):** detectam anéis de fraude e contas laranja pelas relações entre contas, cartões e dispositivos.
- **Sequências (LSTM/Transformer):** avaliam se a transação foge do padrão do próprio cliente.

Desafios: desbalanceamento extremo (fraude < 0,5%; acurácia é inútil, usar precision/recall e PR-AUC), custo assimétrico entre falso negativo e falso positivo, latência de milissegundos, concept drift e explicabilidade regulatória (SHAP).

	Previsão de preços	Detecção de fraude
Sinal	Muito fraco, ruidoso	Forte e claro
Alvo	Incerto, não estacionário	Bem definido (fraude sim/não)
Principal risco	Overfitting e look-ahead bias	Desbalanceamento e drift
Resultado prático	Edge pequeno e frágil	Alto valor comprovado

2. Concept drift: detectar e adaptar

Um modelo com métricas ótimas no deploy pode degradar silenciosamente em semanas, e o rótulo verdadeiro costuma demorar a chegar.

Tipos de mudança

- **Covariate shift:** muda a distribuição das features; a relação X → y continua igual.
- **Concept drift:** muda a própria relação X → y (ex.: padrão legítimo vira assinatura de golpe).
- **Label/prior shift:** muda a proporção das classes (ex.: fraude de 0,2% para 0,6% na Black Friday).
- Quanto ao tempo: abrupta, gradual, incremental ou sazonal/recorrente.

Detecção

- **Entradas (sem rótulo):** PSI (<0,1 estável; 0,1–0,25 atenção; >0,25 mudança significativa), teste KS, divergência KL/JS e *adversarial validation*.
- **Saídas:** distribuição dos scores e taxa de alertas.
- **Desempenho (com rótulo):** precision/recall ao longo do tempo, limitado pelo atraso de rótulo.
- **Detectores online:** ADWIN, DDM/EDDM, Page-Hinkley (biblioteca `river`).

```
def psi(ref, atual, bins=10):
    cortes = np.quantile(ref, np.linspace(0, 1, bins + 1))
    cortes[0], cortes[-1] = -np.inf, np.inf
    p = np.histogram(ref, cortes)[0] / len(ref)
    q = np.histogram(atual, cortes)[0] / len(atual)
    p, q = np.clip(p, 1e-6, None), np.clip(q, 1e-6, None)
    return np.sum((q - p) * np.log(q / p))
```

Adaptação

- **Retreino periódico em janela deslizante:** padrão da indústria; trade-off entre janela curta (adapta rápido, pouca amostra) e longa (estável, lenta).
- **Ponderação por recência:** peso exponencial decrescente em vez de descartar dados antigos.
- **Retreino disparado por evento:** exige mais infraestrutura de MLOps.
- **Aprendizado online:** adapta rápido, mas é sensível a ruído e a dados envenenados.
- **Ensembles dinâmicos:** modelos especialistas por regime com pesos ajustados pelo desempenho recente.
- **Champion/challenger:** challenger em modo sombra, promovido só se superar o champion.

	Fraude	Mercado financeiro
Origem do drift	Adversarial: o fraudador reage ao modelo	Regimes macro, liquidez, política monetária
Velocidade	Rápida (dias/semanas)	Variável, com quebras abruptas
Rótulos	Chegam com atraso	O rótulo é o próprio futuro
Estratégia	Retreino frequente + monitoramento + regras rápidas	Modelos de regime + robustez + gestão de risco

Feedback loop em fraude: ao bloquear uma transação, você nunca sabe se era fraude, e os dados de treino ficam enviesados pelas decisões do próprio modelo. Mitigações: deixar passar uma pequena amostra aleatória com controle de risco e usar *reject inference*.

3. Custo operacional de adaptar o modelo

Em ambiente regulado, computação costuma ser a parte barata. Os custos relevantes são rótulos, engenharia/operação, governança (validação independente pode levar semanas) e o custo de falso alarme.

Regra de decisão: retreinar quando o benefício esperado superar o custo, medindo o drift em unidade de negócio (R$ de fraude escapando), não apenas em PSI.

```
def deve_retreinar(perda_dia_atual, perda_dia_baseline, dias_ate_proximo_ciclo,
                   custo_retreino, prob_melhora=0.7):
    ganho = (perda_dia_atual - perda_dia_baseline) * dias_ate_proximo_ciclo * prob_melhora
    return ganho > custo_retreino
```

Resposta em camadas (da mais barata à mais cara)

Nível	Ação	Custo
1	Ajustar limiar / recalibrar scores	Mínimo
2	Atualizar regras de resposta rápida	Baixo
3	Warm-start com dados recentes	Moderado
4	Retreino completo, mesma arquitetura	Alto
5	Nova arquitetura/features com validação completa	Muito alto

- **Histerese e cooldown:** limiares de entrada/saída e intervalo mínimo entre retreinos.
- **Híbrido calendário + evento:** retreino mensal agendado, com gatilho de exceção só para drift severo.
- **Pré-aprovar o processo, não cada modelo:** governança valida o procedimento (mesma arquitetura, janela e métricas mínimas) e o retreino dentro dos limites passa por checagens automáticas.
- **Champion/challenger barato:** shadow mode, testes sequenciais (SPRT), avaliação só nos casos em que os modelos discordam e rotulagem por active learning.
- **Guardrails de promoção e rollback automático:** transformam o risco de regressão em algo controlado.

Quando vale cada extremo: retreino frequente compensa com custo de erro alto e dinâmica rápida (Pix); retreino raro compensa com modelos caros de validar ou sinais lentos (crédito de longo prazo). No mercado, o custo dominante é o overfitting, não a computação.

4. Overfitting versus perder o timing no mercado

Em mercado é difícil distinguir “o regime mudou” de “foi só ruído”. Com Sharpe verdadeiro de 0,5, são necessários cerca de 4 anos de dados para confiança razoável ($t \approx \text{Sharpe} \times \sqrt{\text{anos}}$). Janela curta ajusta ruído; janela longa treina em regimes que já não existem. É um limite fundamental, não um problema de engenharia.

- **Adaptar poucos parâmetros** (tamanho de posição, exposição, peso entre sinais) em vez do modelo todo.
- **Volatility targeting:** escalar a exposição pela volatilidade recente reage a regimes sem exigir que o modelo preditivo acerte a mudança.
- **Diversificar sinais** fracos e pouco correlacionados; quando um decai, o impacto é diluído.
- **Shrinkage:** $\theta_{final} = \lambda \cdot \theta_{novo} + (1-\lambda) \cdot \theta_{antigo}$, com λ entre 0,2 e 0,3, amortece reações a ruído.
- **Modelos de regime com prior forte** (2–3 regimes com interpretação econômica).
- **Evidência proporcional ao custo de errar:** significância com correção para múltiplos testes (*deflated Sharpe ratio*, López de Prado).

```
vol_alvo = 0.10
vol_recente = retornos.rolling(20).std() * np.sqrt(252)
exposicao = (vol_alvo / vol_recente).clip(upper=2.0)
```

Risco assimétrico: reagir demais gera custo contínuo e silencioso (turnover, spreads, overfitting acumulado); reagir de menos gera custo concentrado em eventos (drawdown de uma quebra de regime). Por isso: **o modelo preditivo muda devagar e com evidência forte; o controle de**

risco muda rápido e mecanicamente. Além disso, parte dos sinais decai porque outros participantes os arbitram, e só pesquisa contínua de novos sinais resolve isso. Sinais com lógica econômica tendem a decair mais devagar.

5. HMM para regimes de mercado: ainda vale?

Sim, mas como componente de risco ou alocação, não como gerador de alpha. Como detector de regime de volatilidade continua sendo uma boa ferramenta, porque o clustering de volatilidade é um dos fatos estilizados mais robustos em finanças.

Onde funciona	Onde falha
Poucos estados (2–3) com interpretação clara; saída probabilística P(regime | dados) para escalar exposição de forma contínua; regimes de volatilidade são persistentes.	**Look-ahead escondido:** `predict_proba` do hmmlearn é *smoothed* (usa o futuro); detecção atrasada em choques abruptos; pressuposto de Markov (duração geométrica); instabilidade e label switching entre refits; escolha do número de estados (BIC pede mais do que a economia sustenta).

Alternativas que ganharam espaço

- **Statistical Jump Models:** penalizam a troca de estado e reduzem o whipsaw.
- **Bayesian change-point detection:** detecta quebras sem fixar o conjunto de regimes.
- **GBDT com features de vol, correlação, spreads e curva de juros:** sem regime explícito.
- **GARCH/EGARCH/GJR:** difíceis de bater em volatilidade com simplicidade comparável.

Para obter a probabilidade **filtrada** (usa só dados até t, como em produção): refit periódico com janela móvel, fixar o rótulo do estado pela variância (evita label switching) e calcular a posterior apenas sobre a janela até t.

Uso	Veredito
Overlay de risco / escala de exposição por regime de vol	Viável e razoável
Alocação entre classes de ativos por regime	Viável, com cuidado e poucos estados
Prever retorno diretamente por regime	Evidência fraca, alto risco de overfitting
Backtest com predict_proba sem filtrar	Resultado enganoso

6. Agentes de IA: monolítico vs. multiagente

Um **agente** é um programa que usa um LLM em loop: o modelo decide a próxima ação, o seu código executa a ferramenta pedida, o resultado volta ao modelo, e o ciclo se repete até haver uma resposta final ou um limite de passos. A diferença entre as arquiteturas está em **quantos agentes** existem e **onde ficam as responsabilidades**.

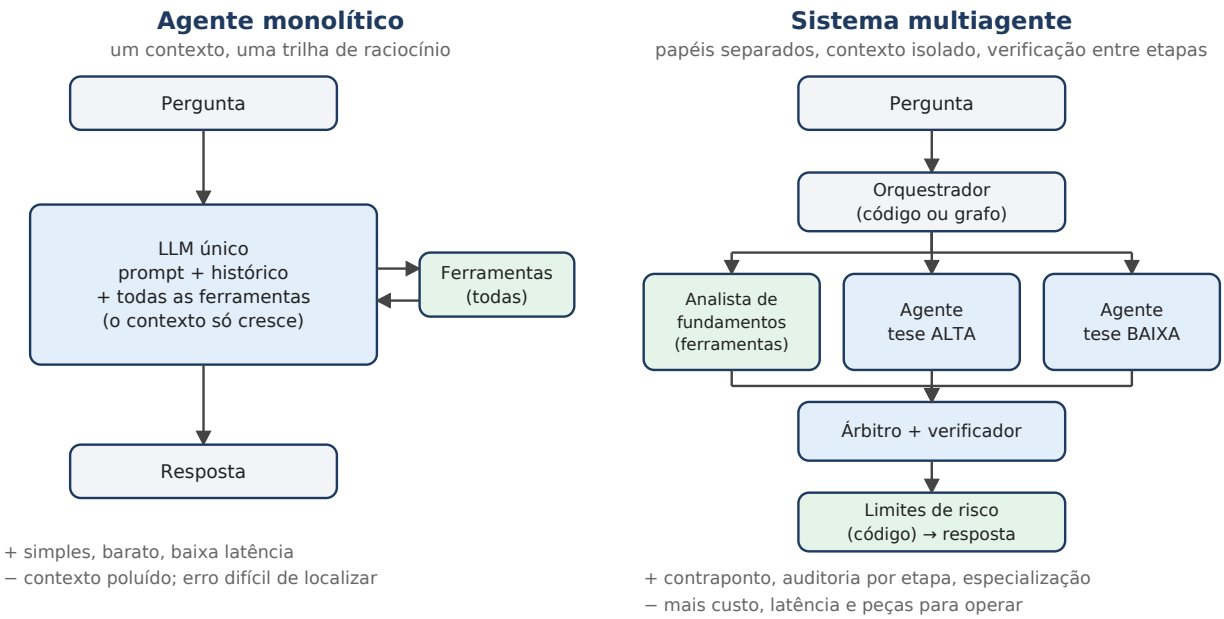


Figura 1. À esquerda, um único LLM com todas as ferramentas e um contexto que cresce. À direita, papéis separados: cada agente enxerga só o que precisa e o risco é uma etapa em código.

	Monolítico	Multiagente
Contexto	Um só; acumula histórico e resultados de ferramentas	Isolado por agente; passa adiante só o necessário (handoff)
Ferramentas	Todas no mesmo agente (mais chance de escolher a errada)	Poucas e específicas por papel
Falha	Um erro no meio contamina o raciocínio inteiro	Dá para validar entre etapas e isolar o erro
Custo e latência	Menores	Maiores (mais chamadas e tokens), mas etapas independentes rodam em paralelo
Depuração	Uma trilha longa e opaca	Trilhas curtas por agente, porém mais peças para operar
Melhor para	Escopo estreito, poucas ferramentas, baixa latência	Várias fases, necessidade de auditoria e contraponto

O que muda na prática em investimentos

- **Contraponto:** um agente monta a tese de alta e outro a de baixa; um árbitro pesa as duas. Isso reduz o viés de confirmação do LLM, que tende a concordar com a própria primeira impressão.
- **LLM só onde ele é bom:** ler relatórios, extrair premissas e sintetizar teses. Cálculos, dados e limites de risco ficam em **código determinístico**; o LLM não faz conta nem decide o tamanho da posição.
- **Verificação de números:** toda evidência citada por um agente é conferida contra a fonte por código. Se não bate, a tese é reprovada antes de chegar ao árbitro (defesa contra alucinação).
- **Auditabilidade:** cada etapa deixa uma saída estruturada (tese, evidências, o que a invalidaria) e um registro, o que permite reconstruir *por que* a recomendação saiu.
- **Veto de risco fora do LLM:** mesmo que o texto do árbitro seja convincente, o limite por ativo e por setor é uma regra em código que reduz ou bloqueia a ordem.

Quando o custo extra compensa

Em decisões **raras e de alto impacto** (mudança estratégica na carteira) o custo por rodada, em tokens, é desprezível diante do que está em jogo; o custo que realmente pesa é o seu tempo de revisão. Por isso peça saídas curtas e estruturadas (tese, evidências com fonte, o que invalidaria a tese). Vale incluir um agente de **pré-mortem** (assume que a decisão deu errado daqui a dois anos e lista as causas prováveis), checklists de concentração, correlação, impacto tributário e liquidez, e conferir o resultado com uma fonte independente. Em tarefas frequentes e simples, um agente único costuma bastar.

Ressalvas: (1) agentes com o mesmo modelo-base têm *erros correlacionados*: o debate dá aparência de diversidade maior do que a real; (2) um erro do primeiro agente vira "fato" para os seguintes se não houver verificação explícita; (3) agentes organizam e sintetizam informação, mas não criam alpha onde o mercado já precificou a notícia; (4) a decisão final e o dimensionamento da posição continuam sendo de quem investe. Regra prática: comece monolítico, meça onde falha e só então divida.

7. Fluxo do pipeline multiagente com veto de risco

O diagrama mostra o pipeline que o código dos capítulos 8 e 9 implementa. Repare em quais etapas usam LLM (azul) e quais são código determinístico (verde): os dados, a verificação dos números e o limite de risco **nunca** dependem da opinião do modelo.

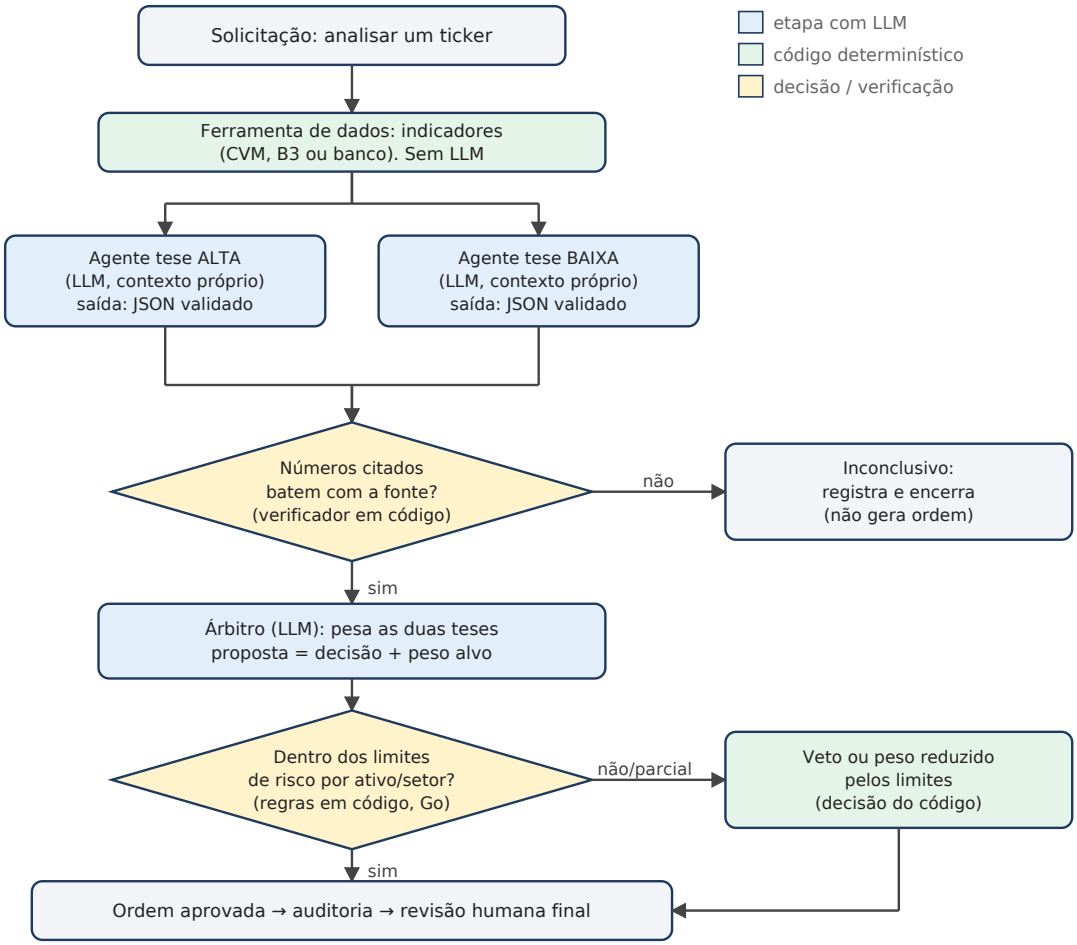


Figura 2. Pipeline de análise de um ticker. As duas teses rodam em paralelo; o verificador e o risco são código e têm poder de barrar o fluxo.

Leitura do fluxo, passo a passo

- **Dados primeiro, por ferramenta.** Os indicadores vêm de uma função, não da "memória" do modelo; assim os números são rastreáveis e reproduzíveis.

- **Tese de alta e de baixa em paralelo.** Cada agente tem um prompt curto e só recebe os dados; ambos devolvem JSON com argumentos, evidências (campo → valor), o que invalidaria a tese e um grau de confiança.
- **Verificador.** Código compara cada evidência com a fonte. Número que não bate ou campo inexistente reprova a análise (resultado “inconclusivo”), em vez de seguir adiante com um dado inventado.
- **Árbitro.** Um terceiro LLM pondera as teses e propõe decisão e peso. É uma *sugestão*.
- **Risco.** O backend aplica limites por ativo e por setor; pode aprovar, reduzir o peso ou vetar. Tudo é registrado na trilha de auditoria e a decisão final passa por uma pessoa.

8. Código Python: os agentes

Arquivo `agentes.py`. Usa o SDK da Anthropic (modelo `claude-sonnet-5-5`), `pydantic` para validar as saídas e `ThreadPoolExecutor` para rodar as duas teses em paralelo. Instalação: `pip install anthropic pydantic fastapi uvicorn`.

Sobre o teste: o backend Go foi compilado, passou em `go vet` e em 3 testes unitários, e foi exercitado ponta a ponta contra o pipeline Python com um **LLM simulado** (lógica do verificador incluída). Neste ambiente não havia acesso ao PyPI, então a chamada real ao SDK da Anthropic e o wrapper FastAPI foram apenas checados por sintaxe: rode-os com a sua chave antes de depender deles.

8.1 Chamada ao modelo e ferramenta de dados

Por quê: isolar a chamada ao LLM em uma única função `llm()` facilita trocar de modelo, medir custo e simular nos testes. A ferramenta `indicadores()` é código comum: devolve sempre os mesmos números para o mesmo ticker (os valores aqui são ilustrativos).

```
"""agentes.py: agente monolítico e pipeline multiagente de análise de ações (didático)."""
import json
from concurrent.futures import ThreadPoolExecutor

import anthropic
from pydantic import BaseModel, Field

MODELO = "claude-sonnet-5-5"
client = anthropic.Anthropic()          # lê ANTHROPIC_API_KEY do ambiente


def llm(system: str, user: str, max_tokens: int = 900) -> str:
    """Uma chamada simples ao modelo, sem ferramentas."""
    r = client.messages.create(
        model=MODELO, max_tokens=max_tokens, system=system,
        messages=[{"role": "user", "content": user}],
    )
    return "".join(b.text for b in r.content if b.type == "text")


# ---------- Ferramenta DETERMINÍSTICA (sem LLM) ----------
def indicadores(ticker: str) -> dict:
    """Na prática: consulta CVM/B3 ou o seu banco. Valores abaixo são ilustrativos."""
    base = {
        "ITUB4": {"p_l": 9.1, "roe": 0.21, "div_liq_ebitda": 0.0, "dy": 0.060},
        "BBAS3": {"p_l": 4.8, "roe": 0.19, "div_liq_ebitda": 0.0, "dy": 0.090},
        "MGLU3": {"p_l": 35.0, "roe": 0.02, "div_liq_ebitda": 3.1, "dy": 0.000},
    }
    return {"ticker": ticker, **base[ticker]}      # KeyError se o ticker não existe
```

8.2 Agente monolítico (um LLM + ferramentas em loop)

Como funciona: o modelo responde ou pede uma ferramenta (`stop_reason == "tool_use"`). Seu código executa a ferramenta, devolve o resultado como `tool_result` e chama o modelo de novo.
Por quê o limite de passos: sem ele, um modelo confuso pode chamar ferramentas indefinidamente e gerar custo sem fim.

```
# ---------- 1) Agente MONOLÍTICO: um LLM + ferramentas em loop ----------
FERRAMENTAS = {"indicadores": indicadores}
TOOLS = [{
```

```
    "name": "indicadores",
    "description": "Retorna P/L, ROE, dívida líquida/EBITDA e dividend yield de um ticker.",
    "input_schema": {"type": "object",
                     "properties": {"ticker": {"type": "string"}},
                     "required": ["ticker"]},
}]


def agente_monolitico(pergunta: str, max_passos: int = 6) -> str:
    msgs = [{"role": "user", "content": pergunta}]
    for _ in range(max_passos):                      # limite evita loop infinito
        r = client.messages.create(model=MODELO, max_tokens=1000,
                                   tools=TOOLS, messages=msgs)
        if r.stop_reason != "tool_use":              # sem ferramenta = resposta final
            return "".join(b.text for b in r.content if b.type == "text")
        msgs.append({"role": "assistant", "content": r.content})
        resultados = []
        for b in r.content:
            if b.type == "tool_use":
                saida = FERRAMENTAS[b.name](**b.input)
                resultados.append({"type": "tool_result", "tool_use_id": b.id,
                                   "content": json.dumps(saida)})
        msgs.append({"role": "user", "content": resultados})
    return "Limite de passos atingido sem resposta final."
```

8.3 Esquemas de saída, agente de tese e verificador

Por quê JSON validado: texto livre não se testa nem se audita; com `pydantic` uma resposta fora do formato gera erro imediato. **Por quê `verificar()` é código:** conferir se "ROE = 0,21" bate com a fonte é uma comparação numérica, e comparação numérica não deve ser delegada a um modelo que pode alucinar.

```
# ---------- 2) Pipeline MULTIAGENTE ----------
class Tese(BaseModel):
    lado: str                                    # "alta" ou "baixa"
    argumentos: list[str]
    evidencias: dict[str, float]                 # campo -> valor citado (será conferido)
    invalida_se: str                             # o que derrubaria a tese
    confianca: float = Field(ge=0, le=1)


class Veredito(BaseModel):
    decisao: str                                 # "comprar" | "manter" | "evitar"
    justificativa: str
    peso_alvo: float = Field(ge=0, le=1)         # fração da carteira PROPOSTA


def _json(texto: str) -> str:
    """Extrai o objeto JSON da resposta (tolera cercas de código)."""
    return texto[texto.find("{"): texto.rfind("}") + 1]


def agente_tese(lado: str, fund: dict) -> Tese:
    sistema = (f"Você é um analista que defende a tese de {lado.upper()}. "
               "Use APENAS os dados fornecidos. Em 'evidencias' cite campo e valor exatos. "
               'Responda só JSON: {"lado","argumentos","evidencias","invalida_se","confianca"}.')
    return Tese.model_validate_json(_json(llm(sistema, json.dumps(fund))))


def verificar(tese: Tese, fund: dict) -> list[str]:
    """Verificador DETERMINÍSTICO: todo número citado precisa bater com a fonte."""
    erros = []
    for campo, valor in tese.evidencias.items():
        fonte = fund.get(campo)
        if not isinstance(fonte, (int, float)):
            erros.append(f"campo inexistente ou não numérico: {campo}")
        elif abs(fonte - valor) > 1e-6 * max(1.0, abs(fonte)):
            erros.append(f"{campo}: citado {valor}, fonte {fonte}")
    return erros
```

8.4 Árbitro e orquestração

Como funciona: `analisar_ticker()` é o orquestrador, escrito como código simples (sem framework): busca os dados, dispara as duas teses em paralelo, verifica, e só então chama o árbitro. Frameworks como LangGraph ou CrewAI fazem o mesmo com um grafo declarativo; para começar, funções comuns são mais fáceis de testar e depurar.

```
def arbitro(alta: Tese, baixa: Tese, fund: dict) -> Veredito:
    sistema = ("Você é o árbitro. Pese as duas teses, diga qual evidência pesou mais "
```

```
            "e proponha uma decisão conservadora. "
            'Responda só JSON: {"decisao","justificativa","peso_alvo"}.')
    entrada = json.dumps({"fundamentos": fund, "alta": alta.model_dump(),
                          "baixa": baixa.model_dump()})
    return Veredito.model_validate_json(_json(llm(sistema, entrada)))


def analisar_ticker(ticker: str) -> dict:
    fund = indicadores(ticker)                        # 1) dados: ferramenta, não LLM
    with ThreadPoolExecutor(max_workers=2) as ex:     # 2) alta e baixa em paralelo
        f_alta = ex.submit(agente_tese, "alta", fund)
        f_baixa = ex.submit(agente_tese, "baixa", fund)
        alta, baixa = f_alta.result(), f_baixa.result()
    erros = verificar(alta, fund) + verificar(baixa, fund)   # 3) confere os números
    if erros:
        return {"ticker": ticker, "decisao": "inconclusivo", "peso_alvo": 0.0,
                "justificativa": "Reprovado na verificação: " + "; ".join(erros)}
    v = arbitro(alta, baixa, fund)                    # 4) árbitro pesa as teses
    return {"ticker": ticker, **v.model_dump(),
            "tese_alta": alta.model_dump(), "tese_baixa": baixa.model_dump()}
```

8.5 Serviço HTTP (FastAPI)

Por quê separar em serviço: o backend em Go não precisa saber nada de LLMs, prompts ou parsing; ele conversa com o pipeline por uma API estável. Erros são traduzidos em códigos HTTP claros: 404 para ticker desconhecido e 502 para falha do LLM ou JSON inválido.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from agentes import analisar_ticker

app = FastAPI()


class Requisicao(BaseModel):
    ticker: str


@app.post("/analisar")
def analisar(req: Requisicao):
    try:
        return analisar_ticker(req.ticker.upper())
    except KeyError:
        raise HTTPException(status_code=404, detail="ticker desconhecido")
    except Exception as e:                  # falha do LLM ou JSON inválido
        raise HTTPException(status_code=502, detail=str(e))

# executar: uvicorn servico_agentes:app --port 8000
```

9. Código Go: o backend

Arquivo `backend/main.go`, só com a biblioteca padrão (Go 1.22 ou superior, por causa das rotas `"POST /analises"` e do `min` embutido). **Por quê Go aqui:** concorrência barata (goroutines), timeouts e cancelamento nativos (`context`), tipagem estática para as regras de risco e um binário único fácil de implantar. O papel do backend é **autoridade**: recebe a proposta dos agentes, aplica os limites, registra e responde.

9.1 Tipos de dados

`Proposta` é o que os agentes sugerem; `Resultado` é o que o backend decide. Separar os dois tipos deixa explícito que a saída do LLM **não é uma ordem**.

```
// Proposta vem do serviço de agentes. É uma SUGESTÃO, nunca uma ordem.
type Proposta struct {
   Ticker        string  `json:"ticker"`
   Decisao       string  `json:"decisao"` // comprar | manter | evitar | inconclusivo
   Justificativa string  `json:"justificativa"`
   PesoAlvo      float64 `json:"peso_alvo"` // fração da carteira a alocar
}

// Resultado é o que o backend decide depois de aplicar as regras de risco.
type Resultado struct {
   Proposta  Proposta `json:"proposta"`
   Aprovada  bool     `json:"aprovada"`
   PesoFinal float64  `json:"peso_final"`
   Motivos   []string `json:"motivos"`
```

```
}

// Limites: regras determinísticas, sem LLM, com poder de veto.
type Limites struct {
   MaxPorAtivo float64 // ex.: 0.05 = 5% da carteira
   MaxPorSetor float64
}

type Carteira struct {
   mu    sync.RWMutex
   pesos map[string]float64 // ticker -> peso atual
   setor map[string]string  // ticker -> setor
}
```

9.2 Regras de risco (determinísticas)

Como funciona: calcula a folga que resta no limite do ativo e do setor e aprova o menor valor entre o pedido e as folgas (`min`). Se não há folga, veta. O `RWMutex` protege a carteira contra acesso concorrente de várias requisições.

```
func (c *Carteira) pesoSetor(setor string) float64 { // exige lock já adquirido
   total := 0.0
   for t, p := range c.pesos {
      if c.setor[t] == setor {
         total += p
      }
   }
   return total
}

func (c *Carteira) Avaliar(p Proposta, lim Limites) Resultado {
   r := Resultado{Proposta: p}
   if p.Decisao != "comprar" {
      r.Motivos = append(r.Motivos, "decisão '"+p.Decisao+"' não gera ordem de compra")
      return r
   }
   if p.PesoAlvo <= 0 || p.PesoAlvo > 1 {
      r.Motivos = append(r.Motivos, "peso_alvo inválido")
      return r
   }
   c.mu.RLock()
   defer c.mu.RUnlock()
   setor, ok := c.setor[p.Ticker]
   if !ok {
      r.Motivos = append(r.Motivos, "ticker sem setor cadastrado")
      return r
   }
   folgaAtivo := lim.MaxPorAtivo - c.pesos[p.Ticker]
   folgaSetor := lim.MaxPorSetor - c.pesoSetor(setor)
   adicional := min(p.PesoAlvo, folgaAtivo, folgaSetor) // min builtin (Go 1.21+)
   if adicional <= 0 {
      r.Motivos = append(r.Motivos, "sem folga nos limites de ativo/setor")
      return r
   }
   if adicional < p.PesoAlvo {
      r.Motivos = append(r.Motivos, fmt.Sprintf(
         "peso reduzido de %.2f%% para %.2f%% pelos limites", p.PesoAlvo*100, adicional*100))
   }
   r.Aprovada, r.PesoFinal = true, adicional
   return r
}
```

9.3 Cliente do serviço de agentes

Por quê `NewRequestWithContext`**:** o contexto carrega o prazo; se o cliente desistir ou o tempo estourar, a chamada ao serviço Python é cancelada, evitando requisições órfãs que continuam gastando tokens. Erros são embrulhados com `%w` para manter a causa.

```
// ClienteAgentes conversa com o serviço Python via HTTP.
type ClienteAgentes struct {
   base string
   http *http.Client
}

func (c *ClienteAgentes) Analisar(ctx context.Context, ticker string) (Proposta, error) {
   var prop Proposta
   corpo, _ := json.Marshal(map[string]string{"ticker": ticker})
   req, err := http.NewRequestWithContext(ctx, http.MethodPost, c.base+"/analisar", bytes.NewReader(corpo))
   if err != nil {
      return prop, err
```

```
    }
    req.Header.Set("Content-Type", "application/json")
    resp, err := c.http.Do(req)
    if err != nil {
        return prop, fmt.Errorf("serviço de agentes: %w", err)
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return prop, fmt.Errorf("serviço de agentes respondeu %d", resp.StatusCode)
    }
    if err := json.NewDecoder(resp.Body).Decode(&prop); err != nil {
        return prop, fmt.Errorf("resposta inválida: %w", err)
    }
    return prop, nil
}
```

9.4 Auditoria, servidor e handler de uma análise

Fluxo do handler: valida a entrada, chama os agentes com timeout de 45 s, aplica o risco, **registra** e responde. A trilha de auditoria aqui é uma lista em memória; em produção seria uma tabela *append-only* no banco, com versão do modelo e das regras.

```
// Auditoria guarda cada decisão (em produção: banco de dados, append-only).
type Auditoria struct {
    mu    sync.Mutex
    itens []map[string]any
}

func (a *Auditoria) Registrar(r Resultado) {
    a.mu.Lock()
    defer a.mu.Unlock()
    a.itens = append(a.itens, map[string]any{"quando": time.Now().UTC(), "resultado": r})
}

type Servidor struct {
    agentes   *ClienteAgentes
    carteira  *Carteira
    limites   Limites
    auditoria *Auditoria
}

func escrever(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}

func (s *Servidor) analisar(w http.ResponseWriter, r *http.Request) {
    var req struct {
        Ticker string `json:"ticker"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil || req.Ticker == "" {
        escrever(w, http.StatusBadRequest, map[string]string{"erro": "ticker obrigatório"})
        return
    }
    ctx, cancel := context.WithTimeout(r.Context(), 45*time.Second)
    defer cancel()
    prop, err := s.agentes.Analisar(ctx, strings.ToUpper(req.Ticker))
    if err != nil {
        log.Printf("erro nos agentes: %v", err)
        escrever(w, http.StatusBadGateway, map[string]string{"erro": err.Error()})
        return
    }
    res := s.carteira.Avaliar(prop, s.limites) // o risco decide, não o LLM
    s.auditoria.Registrar(res)
    escrever(w, http.StatusOK, res)
}
```

9.5 Análise em lote com goroutines

Como funciona: uma goroutine por ticker, com um canal com buffer de 3 posições funcionando como semáforo: só 3 análises rodam ao mesmo tempo, o que respeita o rate limit do provedor de LLM e limita custo. `WaitGroup` espera todas terminarem; cada goroutine escreve no seu próprio índice, então não há corrida de dados. Um erro em um ticker não derruba os outros.

```
// analisarLote roda várias análises em paralelo, com concorrência limitada.
// Atenção: em produção, reservar o peso aprovado de forma transacional para
// que duas análises simultâneas não estourem o mesmo limite.
func (s *Servidor) analisarLote(w http.ResponseWriter, r *http.Request) {
```

```
    var req struct {
        Tickers []string `json:"tickers"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil || len(req.Tickers) == 0 {
        escrever(w, http.StatusBadRequest, map[string]string{"erro": "tickers obrigatório"})
        return
    }
    ctx, cancel := context.WithTimeout(r.Context(), 90*time.Second)
    defer cancel()
    resultados := make([]Resultado, len(req.Tickers))
    erros := make([]string, len(req.Tickers))
    sem := make(chan struct{}, 3) // no máximo 3 simultâneas (custo e rate limit do LLM)
    var wg sync.WaitGroup
    for i, t := range req.Tickers {
        wg.Add(1)
        go func(i int, t string) {
            defer wg.Done()
            sem <- struct{}{}
            defer func() { <-sem }()
            prop, err := s.agentes.Analisar(ctx, strings.ToUpper(t))
            if err != nil {
                erros[i] = err.Error()
                return
            }
            resultados[i] = s.carteira.Avaliar(prop, s.limites)
            s.auditoria.Registrar(resultados[i])
        }(i, t)
    }
    wg.Wait()
    escrever(w, http.StatusOK, map[string]any{"resultados": resultados, "erros": erros})
}
```

9.6 Rotas e função main

```
func (s *Servidor) listarAuditoria(w http.ResponseWriter, r *http.Request) {
    s.auditoria.mu.Lock()
    defer s.auditoria.mu.Unlock()
    escrever(w, http.StatusOK, s.auditoria.itens)
}

func main() {
    base := os.Getenv("AGENTES_URL")
    if base == "" {
        base = "http://localhost:8000"
    }
    s := &Servidor{
        agentes: &ClienteAgentes{base: base, http: &http.Client{Timeout: 60 * time.Second}},
        carteira: &Carteira{
            pesos: map[string]float64{"ITUB4": 0.04},
            setor: map[string]string{"ITUB4": "financeiro", "BBAS3": "financeiro", "MGLU3": "varejo"},
        },
        limites:   Limites{MaxPorAtivo: 0.05, MaxPorSetor: 0.25},
        auditoria: &Auditoria{itens: []map[string]any{}},
    }
    mux := http.NewServeMux()
    mux.HandleFunc("POST /analises", s.analisar)
    mux.HandleFunc("POST /analises/lote", s.analisarLote)
    mux.HandleFunc("GET /auditoria", s.listarAuditoria)
    mux.HandleFunc("GET /saude", func(w http.ResponseWriter, _ *http.Request) {
        escrever(w, http.StatusOK, map[string]string{"status": "ok"})
    })
    srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 * time.Second}
    log.Println("backend em :8080")
    log.Fatal(srv.ListenAndServe())
}
```

9.7 Testes das regras de risco

As regras de risco são a parte que não pode falhar, então são testadas isoladamente, sem rede e sem LLM: redução de peso pelo limite do ativo, veto quando o setor está cheio e ausência de ordem quando a decisão não é de compra.

```
package main

import (
    "math"
    "testing"
)

func novaCarteira() *Carteira {
    return &Carteira{
```

```
      pesos: map[string]float64{"ITUB4": 0.04},
      setor: map[string]string{"ITUB4": "financeiro", "BBAS3": "financeiro"},
   }
}

func TestRiscoReduzPesoPeloLimiteDoAtivo(t *testing.T) {
   r := novaCarteira().Avaliar(
      Proposta{Ticker: "ITUB4", Decisao: "comprar", PesoAlvo: 0.03},
      Limites{MaxPorAtivo: 0.05, MaxPorSetor: 0.25})
   if !r.Aprovada || math.Abs(r.PesoFinal-0.01) > 1e-9 {
      t.Fatalf("esperava aprovar 1%%, veio %+v", r)
   }
}

func TestRiscoVetaQuandoNaoHaFolga(t *testing.T) {
   r := novaCarteira().Avaliar(
      Proposta{Ticker: "BBAS3", Decisao: "comprar", PesoAlvo: 0.03},
      Limites{MaxPorAtivo: 0.05, MaxPorSetor: 0.04}) // setor já está em 4%
   if r.Aprovada {
      t.Fatalf("deveria vetar, veio %+v", r)
   }
}

func TestRiscoIgnoraDecisaoQueNaoEhCompra(t *testing.T) {
   r := novaCarteira().Avaliar(
      Proposta{Ticker: "ITUB4", Decisao: "evitar", PesoAlvo: 0.03},
      Limites{MaxPorAtivo: 0.05, MaxPorSetor: 0.25})
   if r.Aprovada {
      t.Fatalf("não deveria gerar ordem, veio %+v", r)
   }
}
```

10. Arquitetura integrada e como executar

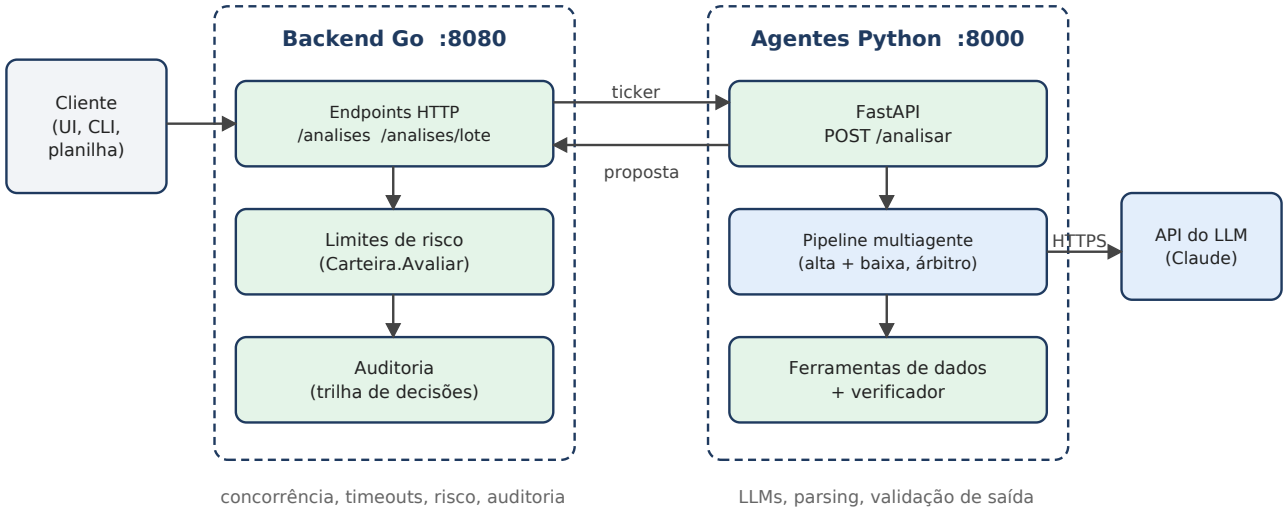


Figura 3. Dois serviços com responsabilidades separadas. O Go é a autoridade (risco e auditoria); o Python concentra tudo que envolve LLM. Só a proposta atravessa a fronteira.

Como rodar

```
# terminal 1: serviço de agentes (Python)
export ANTHROPIC_API_KEY="sua-chave"
pip install anthropic pydantic fastapi uvicorn
uvicorn servico_agentes:app --port 8000

# terminal 2: backend (Go 1.22+)
cd backend
go mod init backend      # só na primeira vez
go run .                 # ouve em :8080

# terminal 3: usar
curl -X POST localhost:8080/analises -d '{"ticker":"ITUB4"}'
curl -X POST localhost:8080/analises/lote -d '{"tickers":["ITUB4","BBAS3"]}'
curl localhost:8080/auditoria
go test ./...            # testes das regras de risco
```

Resultado do teste ponta a ponta (LLM simulado)

Saídas reais do backend rodando contra o pipeline Python com LLM simulado, com carteira inicial de 4% em ITUB4, limite de 5% por ativo e de 25% por setor. Elas mostram o comportamento do **backend**, não a qualidade de uma análise.

Requisição	Resposta do backend
ITUB4 (já em 4%; agentes propõem comprar 3%)	Aprovada com **1%**: “peso reduzido de 3,00% para 1,00% pelos limites”
BBAS3 (sem posição; propõem comprar 3%)	Aprovada com 3%
MGLU3 (ROE baixo; agentes propõem evitar)	Sem ordem: “decisão 'evitar' não gera ordem de compra”
XXXX3 (ticker desconhecido)	Serviço de agentes responde 404; backend responde **502** com mensagem clara
Corpo sem ticker	**400**: “ticker obrigatório”
Lote: BBAS3, MGLU3, XXXX3	Dois resultados válidos e um erro isolado; os outros tickers não são afetados
GET /auditoria	5 registros (as 3 análises individuais e as 2 do lote que chegaram ao risco)

Checklist para levar a produção

- **Reserva transacional de limites:** no lote, duas análises simultâneas podem aprovar o mesmo espaço; reserve o peso no banco dentro de uma transação.
- **Idempotência:** uma chave por solicitação para que repetir a requisição (retry) não gere duas ordens.
- **Retry com backoff e circuit breaker** nas chamadas ao serviço de agentes e ao LLM.
- **Segredos** em cofre ou variáveis de ambiente, nunca no código; **rate limit** na API.
- **Observabilidade:** logs estruturados, métricas (latência, custo em tokens, taxa de reprovação do verificador) e traces entre os dois serviços.
- **Versionar** prompts, modelo e regras de risco; guardar a versão em cada registro de auditoria.
- **Graceful shutdown** do servidor Go e **revisão humana** obrigatória antes de qualquer ordem real.

11. Fluxo de decisão do retreino (drift)

Resume os capítulos 2 e 3: o monitoramento dispara a análise, o impacto é medido em reais e a resposta sobe de nível só se o benefício esperado superar o custo.

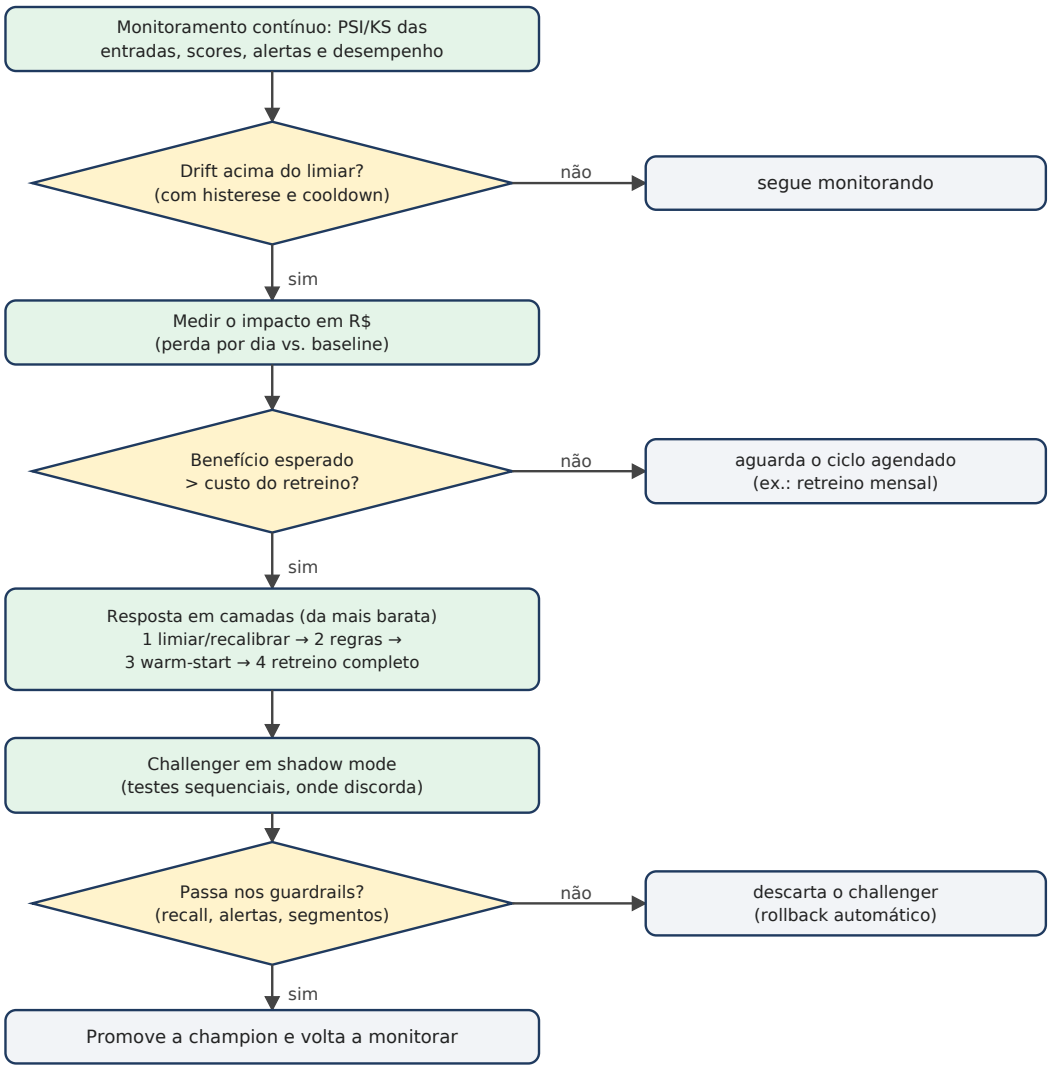


Figura 4. Decisão de retreino: medir em reais, escalar a resposta por camadas e só promover um modelo novo depois de validar em modo sombra.

12. Glossário detalhado

Termos usados neste documento, agrupados por tema.

Agentes e LLMs

Termo	Explicação
LLM (Large Language Model)	Modelo de linguagem treinado em grandes volumes de texto que gera texto a partir de um contexto. É bom em ler, resumir e raciocinar sobre texto; é ruim em aritmética exata e não conhece dados em tempo real sem ferramentas.
Agente	Programa que usa um LLM em loop: o modelo escolhe a próxima ação, o código executa uma ferramenta, o resultado volta ao modelo, e isso se repete até concluir ou atingir um limite de passos.
Agente monolítico	Um único agente (um LLM, um prompt, todas as ferramentas) responsável pela tarefa inteira. Simples e barato, mas o contexto polui e o erro é difícil de localizar.
Sistema multiagente	Vários agentes com papéis, prompts e ferramentas próprios, coordenados por um orquestrador ou grafo. As saídas são combinadas, criticadas ou votadas.
Orquestrador	Componente (código ou LLM) que decide quem executa o quê, em que ordem e como combinar os resultados.
Ferramenta (tool / function calling)	Função que o LLM pode pedir para executar (consultar dados, calcular). O modelo apenas emite um pedido estruturado; quem executa é o seu código, que devolve o resultado.
Contexto / janela de contexto	Todo o texto que o modelo enxerga numa chamada (instruções, histórico, resultados de ferramentas). É limitado, e quanto mais longo, mais detalhes se perdem.
Token	Pedaço de palavra, unidade usada para medir o tamanho da entrada e da saída e para cobrar o uso do modelo.
Prompt / system prompt	Instruções enviadas ao modelo. O system prompt define o papel e as regras do agente (por exemplo, “use apenas os dados fornecidos”).
Saída estruturada	Obrigar o modelo a responder em formato fixo (JSON) que o código valida, em vez de texto livre. Torna a resposta testável e auditável.
Alucinação	Quando o modelo produz uma informação plausível, porém falsa ou não sustentada pelos dados. Mitiga-se com ferramentas, citação de fonte e verificação determinística.
Guardrail	Regra que limita o que o sistema pode fazer, idealmente em código fora do LLM (limite de posição, lista de ativos permitidos, recall mínimo para promover um modelo).
Veto	Poder de uma etapa (como o risco) de bloquear ou reduzir a decisão proposta pelas etapas anteriores.
Human-in-the-loop	Ponto do fluxo em que uma pessoa revisa ou aprova antes de a ação ter efeito.
Handoff	Passagem do resultado de um agente a outro, levando só o necessário e em formato estruturado, para não poluir o contexto do próximo.
Grafo de execução / DAG	Fluxo modelado como nós (etapas) e arestas (ordem). Permite paralelismo e ramificações condicionais. DAG é um grafo dirigido sem ciclos. LangGraph é uma biblioteca que implementa essa ideia para agentes.
RAG (Retrieval-Augmented Generation)	Técnica de recuperar trechos relevantes de documentos (relatórios, balanços) e entregá-los ao modelo no contexto, em vez de depender do que ele memorizou.
Erros correlacionados	Agentes baseados no mesmo modelo tendem a errar do mesmo jeito. Ter vários agentes não garante opiniões independentes.
Red team / pré-mortem	Atacar a própria tese de propósito. No pré-mortem assume-se que a decisão falhou no futuro e listam-se as causas mais prováveis, antes de decidir.

Engenharia: APIs, Python e Go

Termo	Explicação
API / endpoint	Interface por onde programas conversam, em geral via HTTP. Endpoint é uma rota específica, como POST /analises.
JSON	Formato de texto para troca de dados estruturados entre serviços.
Timeout	Tempo máximo de espera por uma chamada. Evita que o sistema trave esperando um serviço lento.
context.Context (Go)	Objeto que carrega prazo (deadline) e sinal de cancelamento entre funções e goroutines. Ao cancelar, as chamadas em andamento são interrompidas.
Goroutine	"Thread leve" do Go, gerenciada pelo runtime. Milhares podem rodar de forma concorrente com baixo custo.
Canal (channel)	Fila tipada para goroutines se comunicarem. Um canal com buffer fixo também serve de semáforo para limitar a concorrência.
WaitGroup / Mutex	WaitGroup espera um conjunto de goroutines terminar. Mutex (ou RWMutex) garante que só uma goroutine altere um dado compartilhado por vez, evitando corrida de dados.
Concorrência vs. paralelismo	Concorrência é lidar com várias tarefas em andamento; paralelismo é executá-las de fato ao mesmo tempo em vários núcleos. Chamadas a LLM são I/O: a concorrência já ajuda muito.
FastAPI / uvicorn	Framework web e servidor Python usados para expor o pipeline de agentes como API.
Pydantic	Biblioteca Python que valida dados contra um esquema tipado e dá erro claro quando a saída do LLM foge do formato.
Idempotência	Propriedade de uma operação que, repetida, produz o mesmo efeito. Essencial para que um retry não crie duas ordens.
Retry com backoff / circuit breaker	Retry reenvia a chamada que falhou, com espera crescente (backoff). O circuit breaker interrompe as chamadas a um serviço que está falhando, evitando efeito cascata.
Rate limit	Limite de requisições por intervalo de tempo imposto pelo provedor da API (por exemplo, do LLM).
Trilha de auditoria (audit log)	Registro imutável de quem decidiu o quê, com entradas, saídas, versão do modelo e das regras. Permite reconstruir uma decisão depois.
Observabilidade	Conjunto de logs, métricas e traces que permite entender o que o sistema fez e por que.
Graceful shutdown	Encerrar o servidor terminando as requisições em andamento antes de parar.

Dados, drift e MLOps

Termo	Explicação
Drift	Mudança ao longo do tempo na distribuição dos dados ou na relação entre entradas e saída, que degrada o desempenho de um modelo.
Covariate shift	Muda a distribuição das features, mas a relação X → y continua a mesma (ex.: mais transações via Pix e menos cartão).
Concept drift	Muda a própria relação X → y (ex.: um padrão antes legítimo vira assinatura de golpe).
Prior / label shift	Muda a proporção das classes (ex.: a taxa de fraude sobe de 0,2% para 0,6%).
PSI (Population Stability Index)	Mede o quanto a distribuição de uma variável mudou entre uma referência e o período atual. Regra prática: abaixo de 0,1 estável; de 0,1 a 0,25 atenção; acima de 0,25 mudança significativa.
KS (Kolmogorov-Smirnov)	Teste estatístico que compara duas distribuições e indica se são diferentes.

Termo	Explicação
ADWIN, DDM, Page-Hinkley	Detectores online de mudança: acompanham o fluxo de erros e disparam quando a taxa de erro muda de forma estatisticamente relevante (biblioteca river).
Adversarial validation	Treinar um classificador para distinguir dados antigos de novos; se ele acerta bem, as distribuições mudaram.
Atraso de rótulo	O valor verdadeiro só é conhecido depois (contestação de fraude, retorno de 20 dias). Por isso se monitoram entradas e saídas antes do desempenho.
Feedback loop	As decisões do modelo alteram os dados futuros: transações bloqueadas nunca revelam se eram fraude, enviesando o treino.
Reject inference	Técnicas para estimar o resultado de casos que o modelo rejeitou e que, portanto, nunca foram observados.
Champion / challenger	O modelo em produção (champion) é comparado com um candidato (challenger), que só o substitui se for melhor.
Shadow mode	O challenger recebe os mesmos dados e gera previsões, mas não toma decisões. Custa só inferência e não oferece risco.
SPRT (teste sequencial)	Teste que decide assim que há evidência suficiente, em vez de esperar uma janela fixa, encurtando a avaliação do challenger.
Active learning	Priorizar a rotulagem dos casos mais incertos ou informativos, para gastar menos com analistas.
Warm-start	Continuar o treino a partir de um modelo já treinado, usando dados recentes, em vez de começar do zero.
Histerese e cooldown	Histerese usa limiares diferentes para ligar e desligar um gatilho; cooldown impõe um intervalo mínimo entre ações. Evitam retreinos em cascata quando um indicador oscila perto do limite.
MLOps	Práticas e ferramentas para colocar modelos em produção e mantê-los: pipelines, monitoramento, versionamento, retreino e rollback.

Mercado e validação

Termo	Explicação
B3 / CVM	B3 é a bolsa de valores brasileira; CVM é o regulador do mercado de capitais e fonte de demonstrações financeiras das companhias abertas.
P/L, ROE, dívida líquida/EBITDA, dividend yield	P/L: preço dividido pelo lucro por ação. ROE: lucro sobre patrimônio líquido. Dívida líquida/EBITDA: alavancagem em anos de geração de caixa operacional. Dividend yield: proventos sobre o preço.
Look-ahead bias	Usar, no treino ou no backtest, informação que ainda não estaria disponível no momento da decisão. Infla resultados de forma enganosa.
Walk-forward	Validação em que se treina no passado, testa no período seguinte e avança a janela no tempo, imitando o uso real.
Purging / embargo	Remover do treino as observações próximas ao período de teste, cujo alvo se sobrepõe no tempo, para evitar vazamento de informação.
Survivorship bias	Analisar só ativos que sobreviveram, ignorando os que saíram do mercado, o que superestima o retorno.
Overfitting	O modelo se ajusta ao ruído dos dados de treino e não generaliza para dados novos.
Backtest	Simulação de uma estratégia com dados históricos. Só é confiável com custos realistas e validação temporal correta.
Slippage / turnover	Slippage é a diferença entre o preço esperado e o executado. Turnover é o quanto a carteira é negociada; ambos corroem o retorno.

Termo	Explicação
Edge	Vantagem estatística pequena e persistente de uma estratégia, que é o que sobra depois de custos.
Não-estacionariedade / regime	As propriedades do mercado mudam com o tempo. Regime é um período com comportamento estatístico próprio (baixa volatilidade, alta volatilidade, crise).
Volatilidade e clustering de volatilidade	Volatilidade mede a dispersão dos retornos. O clustering é a tendência de períodos agitados seguirem períodos agitados, um dos fatos mais robustos em finanças.
Drawdown	Queda do pico ao vale do valor de uma carteira ou estratégia.
Sharpe e deflated Sharpe	Sharpe é o retorno em excesso dividido pela volatilidade. O deflated Sharpe corrige o valor para o número de estratégias testadas, reduzindo falsos achados por múltiplos testes.
Volatility targeting	Escalar a exposição pela volatilidade recente para manter o risco aproximadamente constante, sem precisar prever a mudança de regime.
Shrinkage	Combinar o parâmetro novo com o antigo ($\theta_{final} = \lambda \cdot \theta_{novo} + (1-\lambda) \cdot \theta_{antigo}$) para amortecer reações a ruído.
HMM (Hidden Markov Model)	Modelo em que um estado oculto (o regime) muda segundo probabilidades de transição e gera as observações. Serve para estimar a probabilidade de cada regime.
Filtered vs. smoothed	Probabilidade filtrada usa só dados até o instante t (o que existe em produção). A suavizada usa também o futuro e, em backtest, cria um viés de look-ahead.
Label switching / whipsaw	Label switching: o rótulo dos estados do HMM muda entre treinos. Whipsaw: trocas rápidas de sinal que geram perdas por oscilar em torno do limite.
Information coefficient	Correlação entre o sinal previsto e o retorno realizado; acompanhar seu decaimento mostra quando um sinal perde força.

Modelos e métricas

Termo	Explicação
GBDT (XGBoost, LightGBM)	Gradient Boosted Decision Trees: muitas árvores pequenas treinadas em sequência, cada uma corrigindo os erros das anteriores. Referência em dados tabulares.
GARCH	Família de modelos de séries temporais para a volatilidade que muda no tempo (EGARCH e GJR são variantes).
LSTM, GRU, Transformer	Redes neurais para sequências. Os Transformers usam atenção para relacionar pontos distantes da série.
GNN (rede neural em grafos)	Modelo que aprende sobre nós e arestas, útil para detectar anéis de fraude pelas relações entre contas, cartões e dispositivos.
Isolation Forest, autoencoder, One-Class SVM	Métodos de detecção de anomalias sem rótulos: isolam pontos raros, medem erro de reconstrução ou delimitam a região “normal”.
Precision, recall, PR-AUC	Precision: dos alertas, quantos eram fraude. Recall: das fraudes, quantas foram pegas. PR-AUC resume o equilíbrio entre os dois e é mais informativa que a acurácia em dados desbalanceados.
Desbalanceamento, SMOTE, class weights	Quando uma classe é rara. SMOTE cria exemplos sintéticos da classe rara; class weights dão mais peso a ela na perda.
SHAP	Método que atribui a cada feature sua contribuição para uma previsão específica, usado para explicar por que uma transação foi barrada.
Aprendizado por reforço	O agente aprende uma política de ações maximizando uma recompensa (por exemplo, retorno ajustado ao risco). Promissor, porém instável e propenso a overfitting.

13. Próximos passos sugeridos

- **Laboratório de drift:** simular um fluxo de transações com drift injetado e comparar modelo estático, janela deslizante, ADWIN + retreino e a política em camadas pelo **custo total** (perda de negócio + custo de retreino).
- **Laboratório de mercado:** troca de regime simulada comparando janelas de tamanhos diferentes, shrinkage e volatility targeting por Sharpe e drawdown, com HMM *smoothed* versus *filtered*.
- **Agentes:** rodar o pipeline deste documento com a sua chave e dados reais da B3/CVM, e comparar a versão monolítica com a multiagente em uma tarefa concreta, medindo custo, latência e taxa de reprovação do verificador para ver onde dividir em agentes realmente ajuda.